

DORA Metrics Cheat Sheet

23 tables · 145 concepts · Updated 2026-05-28 · [cheatgrid.com](#)

DORA (DevOps Research and Assessment) is a research program that defines five metrics for measuring software delivery performance. Its research began in 2014, and in 2024 the framework evolved to the current five-metric form, grouped as **throughput** (Change Lead Time, Deployment Frequency, Failed Deployment Recovery Time) and **instability** (Change Fail Rate, Deployment Rework Rate). The 2025 DORA report, focused on AI-assisted software development, established the central insight that **AI is an amplifier**: it accelerates strong teams and magnifies dysfunction in struggling ones, making solid engineering foundations matter more than ever. One enduring DORA finding still holds: **speed and stability are not tradeoffs**, since elite teams excel at both, consistently showing that practices enabling frequent deployment also improve reliability.

CONTENTS

Core DORA Metrics	2
Performance Benchmarks by Tier	2
Deployment Frequency Measurement	3
Lead Time Stages and Breakdown	3
Change Failure Rate Calculation	3
Recovery Time Measurement	4
Metric Collection Automation	4
Dashboard Design Best Practices	5
Team-Level vs Organization-Level Metrics	5
Common Implementation Pitfalls	6
Avoiding Metrics Gaming	6
Metrics-Driven Improvement Strategies	7
Correlation with Business Outcomes	7
Throughput vs Instability Balance	8
Research Methodology and Evidence	8
Trend Analysis Over Time	9
Qualitative vs Quantitative Indicators	9
Tools and Platforms	10
Platform Engineering Impact	10
Organizational Culture and Westrum Model	11
Advanced Topics and Extensions	11
DORA AI Capabilities Model	12
AI-Era DORA Extensions	12
References	13

TABLE 1

Core DORA Metrics

The five DORA metrics form an interconnected system covering two factors: throughput (how fast changes flow to production) and instability (how often those changes cause problems). Tracking all five together prevents the gaming that arises from optimizing a single number.

Metric	Example	Description
Deployment Frequency	Multiple deploys/day (elite)	- How often code is deployed to production throughput metric measuring team velocity and continuous delivery maturity.
Change Lead Time	Commit at 9am → Production at 11am = 2 hours	- Time from code commit to running in production - measures end-to-end delivery speed including coding, review, testing, and deployment.
Failed Deployment Recovery Time	Incident detected at 2pm, service restored at 3:30pm = 1.5 hours	- Time to restore service after a deployment-caused failure - renamed from MTTR in 2023 to focus strictly on change-induced outages, not external infrastructure failures; throughput metric.
Change Fail Rate	5 deployments, 1 requires hotfix = 20%	- Percentage of production deployments requiring immediate remediation (rollback, hotfix, or incident) instability metric reflecting deployment quality.
Deployment Rework Rate	3 out of 20 deploys this month were unplanned incident fixes = 15%	- Ratio of deployments that are unplanned work caused by production incidents rather than new feature releases - added as the official 5th metric in 2024 to capture reactive capacity drain; instability metric.

TABLE 2

Performance Benchmarks by Tier

DORA's four performance tiers — derived from statistical cluster analysis of thousands of teams — remain the standard for benchmarking each metric. The 2025 DORA report also introduced seven team archetypes (see Table 15) that capture the interplay of performance, stability, and well-being beyond simple tier labels.

Tier	Example	Description
Elite	Multiple deploys/day <1 day lead time 0-15% failure rate <1 hour FDRT	- Top performers with on-demand deployments, sub-day delivery, minimal failures, and rapid recovery - top ~19% of surveyed teams.
High	Daily to weekly deploys <1 week lead time 16-30% failure rate <1 day FDRT	Strong performers with regular deployment cadence , weekly delivery cycles, and same-day incident resolution.
Medium	Weekly to monthly deploys 1 week to 1 month lead time 16-30% failure rate <1 week FDRT	Average performers with periodic releases , longer feedback loops, and multi-day recovery times.
Low	Less than monthly deploys >1 month lead time 46-60% failure rate >1 week FDRT	Bottom quartile with infrequent releases , extended delivery cycles, high failure rates, and prolonged outages.

TABLE 3

Deployment Frequency Measurement

Deployment frequency is straightforward in concept but subtle in practice — the key is counting only what genuinely reaches end users, and choosing the right aggregation method to avoid distortion from outliers or quiet periods.

Approach	Example	Description
Per-day calculation	100 deployments ÷ 30 days = 3.33/day	- Count production deployments over time period simplest calculation dividing total deploys by days measured.
On-demand classification	Team deploys whenever ready, averaging 5x/day	- Elite teams deploy multiple times daily without fixed schedules - indicates mature CI/CD and automated testing.
Production-only counting	Exclude staging/dev deploys, count prod only	- Only deployments reaching end users count - staging and test environment deploys are excluded.
Median vs mean	Median: 1/day (ignores outliers) Mean: 2.5/day (includes spikes)	Median preferred over mean to avoid skewing from irregular batch releases or quiet periods.
Time-between calculation	Deploy Mon, deploy Thu = 3 days between	Alternative approach measuring average days between successive deployments rather than deploys per period.

TABLE 4

Lead Time Stages and Breakdown

Lead time for changes is rarely limited by coding speed — most of the clock ticks during waiting: idle queues before review, slow CI pipelines, and delayed deployment windows. Decomposing the total into stages reveals exactly where the bottleneck lives.

Stage	Example	Description
Coding time	First commit → Last commit on feature = 4 hours	- Active development time from initial commit to completion - excludes waiting periods.
Review time	PR opened → PR approved = 8 hours	Code review duration including wait time for reviewers and addressing feedback - bottleneck in many teams; often expands in AI-assisted environments as review volume increases.
Testing time	CI pipeline runs automated tests = 15 minutes	- Automated and manual testing duration - includes CI/CD pipeline execution time.
Deployment time	Merge to main → Deployed to prod = 30 minutes	Production deployment execution including artifact building, environment provisioning, and release automation.
Waiting time	Completed code sits 12 hours before review starts	Idle time between stages - often the largest component of lead time and prime improvement target.
Total lead time	First commit → Running in production = 2 days	End-to-end time from code commit to production - sum of all stages including waiting periods.

TABLE 5

Change Failure Rate Calculation

Change fail rate is the most debated DORA metric because "failure" requires a team definition. Consistent, agreed-upon definitions matter far more than a precise number — a well-defined 20% CFR is more actionable than a vague 5%.

Method	Example	Description
Basic formula	(1 failure ÷ 5 deploys) × 100 = 20%	- Failed deployments divided by total deployments percentage of changes requiring immediate remediation.
Failure definition	Deployment causes degraded service or hotfix needed	- Failure requires rollback, hotfix, or incident response - minor bugs found later may not count.

Method	Example	Description
Rollback detection	Version 1.5 deployed, then 1.4 redeployed = rollback	Automatically detect when previous version is redeployed shortly after new version.
Incident correlation	SEV1 incident within 24h of deploy links to that deploy	Link production incidents to recent deployments within detection window (typically 24-48 hours).
Hotfix counting	Emergency patch deployed same day = failed change	Count unplanned remediation deployments following a regular release as failures.
Time window	Issues appearing within 48h of deploy count as failures	• Define detection window for linking failures to deployments • too short misses issues, too long inflates rate.

TABLE 6

Recovery Time Measurement

Failed deployment recovery time (formerly MTTR) focuses strictly on recovery from deployment-caused failures, not all production incidents. This distinction matters: infrastructure outages should not inflate the metric that measures delivery quality.

Strategy	Example	Description
Incident timestamp	Alert fired 2pm, resolved 3:30pm = 90 minutes	• Measure from incident detection (first alert/ticket) to resolution (service restored) • most common approach.
User-impact focus	Users affected 2:05pm, service normal 3:25pm = 80 minutes	• Time between user-facing impact beginning and ending • more accurate than internal detection timestamps.
Detection to fix	Issue detected → Code fix deployed = 45 minutes	• Narrow definition measuring fix deployment time only • excludes detection lag and validation.
Median calculation	10 incidents: 20m, 30m, 1h, 2h... → median = 50m	• Use median recovery time rather than mean • prevents single long outages from distorting metric.
Severity weighting	SEV1 incidents tracked separately from SEV3	• Track recovery time by incident severity • critical outages and minor issues have different expectations.
Deployment-only scope	Only count failures caused by your own code changes	• Exclude infrastructure outages and third-party failures from the metric • the 2023 DORA redefinition explicitly scoped this to deployment-caused incidents.

TABLE 7

Metric Collection Automation

Manual data collection for DORA rarely scales beyond a team of five — automation is essential. The key is instrumenting pipelines at the right points so events flow to your metrics system without requiring developer overhead.

Tool	Example	Description
CI/CD pipeline tags	GitHub Actions logs deployment timestamp to database	• Instrument pipeline stages to emit structured events • timestamp commits, builds, tests, and deployments.
Git webhook integration	GitLab sends commit events to metrics collector API	Use repository webhooks to capture commit timestamps and PR merge events automatically.
Incident management API	PagerDuty API queries incidents linked to deployments	Pull incident data from ticketing systems (Jira, PagerDuty, Opsgenie) to calculate CFR and FDRT.
OpenTelemetry tracing	OTEL spans track code from commit through production	• Use distributed tracing to measure lead time across stages • captures timing automatically.
Apache DevLake	DevLake connects GitHub + Jira + Jenkins → DORA dashboard	• Open-source DORA platform aggregating data from Git, CI/CD, and incident tools • strongest self-hosted option; ships with a built-in Grafana dashboard.

Tool	Example	Description
Platform metric APIs	Datadog DORA metrics dashboard queries deployment logs	• Leverage observability platforms with built-in DORA tracking • Datadog, New Relic, Splunk offer native support.
Custom collector scripts	Python script queries Git + Jenkins + ServiceNow hourly	• Build custom integration pulling data from multiple sources • useful when tools lack native DORA support.

TABLE 8

Dashboard Design Best Practices

A good DORA dashboard surfaces actionable signals, not just metrics — the design choices around context, comparison, and drill-down determine whether a dashboard drives improvement or just decorates a wall.

Practice	Example	Description
All five metrics together	Single view showing DF, LT, CFR, FDRT, DRR side-by-side	• Display all metrics simultaneously to show throughput/instability balance • avoids tunnel vision on one dimension.
Trend lines over time	30-day rolling average with week-over-week comparison	• Show historical trends not just current values • reveals improvement/regression patterns over time.
Performance tier indicators	Color-coded badges: Elite (green), High (blue), Medium (yellow), Low (red)	• Visually indicate benchmark tier using DORA research categories • makes performance level immediately clear.
Drill-down capability	Click CFR → View list of failed deployments with details	• Enable detailed exploration from summary metrics • users can investigate specific incidents or slow deployments.
Team vs org views	Toggle between team-level and company-wide rollups	• Support multiple aggregation levels • individual teams and organizational leaders have different needs.
Goal tracking	Progress bar: Current 2/day → Target 5/day deployment frequency	• Display improvement targets alongside current values • keeps teams aligned on objectives.

TABLE 9

Team-Level vs Organization-Level Metrics

Aggregation strategy is one of the most important — and most overlooked — DORA decisions. Rolling up metrics too broadly hides the teams that need help and the teams worth learning from.

Scope	Example	Description
Team-level tracking	Mobile team: 8/day DF Backend team: 3/day DF	• Measure individual team performance • avoids averaging out high/low performers and enables targeted improvement.
Service-level tracking	User-service: <1h LT Payment-service: 4h LT	• Track metrics per service/microservice • reveals which components have efficient vs slow delivery.
Organization aggregate	Company-wide: 500 deploys/week across 20 teams	• Roll up to company-wide totals • useful for executive reporting and cross-team benchmarking.
Avoid forced averaging	Don't average 1 elite + 1 low team = medium performance	• Distribution matters more than mean • showing performance spread across teams is more valuable than single number.
Team comparison	Compare similar teams, not frontend vs infrastructure	• Compare teams with similar contexts • web app teams vs infrastructure teams face different constraints.

TABLE 10

Common Implementation Pitfalls

Most DORA implementations fail not from technical complexity but from definitional inconsistency, missing baselines, or misuse as competitive rankings rather than improvement guides.

Pitfall	Example	Description
Measuring without baseline	Start tracking CFR with no historical reference point	- Establishing initial baseline is critical - without it, impossible to determine if improvements are working.
Inconsistent definitions	Team A counts staging deploys, Team B only counts production	Standardize metric definitions across teams - inconsistent counting makes comparisons meaningless.
Gaming the metrics	Split one deploy into five to inflate deployment frequency	- Goodhart's Law: when measure becomes target, ceases to be good measure - focus on outcomes not numbers.
Ignoring context	Blame team for low DF when they're maintaining legacy system	- Consider organizational constraints - regulated industries, legacy tech, and compliance have legitimate friction.
Cherry-picking metrics	Only report deployment frequency, hide change failure rate	- Track all five metrics as a set - optimizing one while ignoring others creates false performance picture.
Tool dependency	Assume Azure DevOps tracks DORA automatically without configuration	- Most tools require custom configuration - default dashboards rarely track DORA without pipeline instrumentation.
Siloed ownership	Dev team owns DF, ops team owns FDRT with no shared discussion	- Metrics must be shared across dev, ops, and release teams - isolated ownership creates finger-pointing rather than collaboration.

TABLE 11

Avoiding Metrics Gaming

Gaming emerges when metrics become targets rather than signals. The antidote is a combination of multiple counter-balanced metrics, qualitative checks, and a culture where improvement matters more than hitting a number.

Strategy	Example	Description
Focus on outcomes	Deploy to deliver value, not to hit a deployment count	- Emphasize customer impact and business outcomes as primary goals - metrics guide but don't define success.
Balanced scorecard	Elite DF but high CFR reveals quality problems	- View metrics holistically - speed without stability or stability without speed indicates imbalance.
Qualitative checks	Survey: "Do deployments feel risky?" despite low CFR	- Supplement quantitative DORA metrics with qualitative feedback - numbers don't capture fear, toil, or satisfaction.
No individual targeting	Never use DORA for performance reviews or ranking devs	- DORA measures team/system performance not individuals - using for personal evaluation destroys psychological safety.
Celebration over punishment	Highlight improvement, not blame for low performers	- Use metrics for learning and improvement - punishment creates metric manipulation and fear.
System thinking	Low DF reveals slow review process, not lazy developers	- Investigate systemic bottlenecks when metrics lag - usually indicates process/tool issues not people problems.

TABLE 12

Metrics-Driven Improvement Strategies

The highest-leverage improvements typically address multiple DORA metrics simultaneously — automated testing, small batch sizes, and trunk-based development each improve throughput and stability at once, compounding returns.

Strategy	Example	Description
Small batch sizes	Deploy single feature vs months of accumulated changes	• Smaller changes are easier to test, review, and roll back • improves all five DORA metrics; especially critical in AI-assisted environments where large AI-generated PRs slow reviews.
Automate testing	Add comprehensive unit/integration test suite	• Automated test coverage enables confident frequent deployments • reduces CFR and improves DF simultaneously.
Trunk-based development	All devs commit to main, use feature flags for incomplete work	• Short-lived branches and frequent integration reduce lead time • decreases merge conflicts and accelerates feedback.
Progressive delivery	Canary deploy to 5% → 25% → 100% with automated rollback	• Gradual rollouts with monitoring catch issues before full impact • reduces FDRT and improves CFR.
Deployment automation	One-click production deploys via CI/CD pipeline	• Remove manual deployment steps • reduces lead time, increases DF, and minimizes human error causing failures.
Observability investment	Add structured logging, metrics, tracing to all services	• Real-time monitoring enables fast failure detection and diagnosis • directly improves FDRT.
Blameless postmortems	After incident, focus on process improvements not blame	• Learning from failures without punishment • improves both CFR and FDRT through systemic fixes.

TABLE 13

Correlation with Business Outcomes

DORA research has repeatedly shown that high software delivery performance predicts better organizational outcomes — not just faster features, but higher revenue growth, stronger market position, and better employee retention.

Outcome	Example	Description
Faster time to market	Low LT enables quick response to competitor features	Rapid delivery lets organizations capitalize on market opportunities before competitors.
Higher customer satisfaction	Frequent small releases reduce disruptive big-bang updates	• Continuous improvement delivers value faster and with less disruption • increases user retention.
Reduced operational costs	Low FDRT minimizes revenue loss during outages	• Fast recovery limits business impact of incidents • every hour of downtime has direct financial cost.
Increased innovation	Elite teams spend less time firefighting, more on new features	• Lower failure rates free engineering capacity • teams can focus on innovation vs operational toil.
Competitive advantage	2x faster deployment enables A/B testing and experimentation	• High throughput with stability allows rapid learning • organizations can test hypotheses and adapt quickly.
Employee retention	Elite-performing teams have higher job satisfaction	• Better metrics correlate with better work experience • less toil and firefighting improves morale.

TABLE 14

Throughput vs Instability Balance

The DORA framework deliberately splits metrics into throughput and instability rather than speed and quality — the framing matters because instability is an outcome of delivery practices, not a separate quality axis. Teams that improve throughput without controlling instability are not actually improving.

Concept	Example	Description
Not a tradeoff	Elite teams: High DF + Low CFR simultaneously	- DORA research consistently shows speed and stability improve together - practices enabling one benefit both.
Throughput metrics	Deployment Frequency + Change Lead Time + Failed Deployment Recovery Time	- Measure how fast and how much teams deliver - velocity, flow efficiency, and recovery speed through the delivery pipeline.
Instability metrics	Change Fail Rate + Deployment Rework Rate	- Measure how reliably teams deliver - production quality and proportion of reactive vs planned work.
Correlation pattern	Teams improving DF typically also improve CFR	- Metrics are positively correlated not inversely - automation and testing improve all dimensions.
False speed	High DF with 50% CFR = thrashing, not true velocity	Deployment frequency alone misleads - frequent failed releases waste more time than slower quality releases.
False stability	Low CFR with monthly deploys = avoiding risk, not quality	Low failure rate with infrequent deploys may indicate fear of change not robust processes.
High rework signal	High DF + High DRR = shipping fast but creating debt	Deployment rework rate rising alongside frequency signals reactive capacity drain - the team is moving fast but generating more unplanned work than value.

TABLE 15

Research Methodology and Evidence

DORA's credibility rests on its empirical foundation — statistical analysis of survey data from tens of thousands of practitioners, peer-reviewed in the Accelerate book, and continuously refined across more than a decade of annual reports.

Aspect	Example	Description
Survey-based data	Annual State of DevOps Report surveys 30,000+ practitioners	- DORA research uses large-scale surveys across industries - statistically rigorous analysis identifies patterns.
Cluster analysis	Statistical clustering identifies performance groups, not fixed thresholds	- Performance tiers emerge from data-driven clustering not arbitrary cutoffs - benchmarks evolve as industry improves.
Accelerate book	Nicole Forsgren, Jez Humble, Gene Kim (2018)	Foundational research publication establishing DORA framework - demonstrates statistical relationships between practices and outcomes.
Seven team archetypes	2025 report: Foundational Challenges → Harmonious High-Achievers	2025 DORA replaced simple 4-tier ranking with 7 team archetypes combining delivery, stability, and well-being - profiles include Foundational Challenges, Legacy Bottleneck, Process-Constrained, Pragmatic Performers, and Harmonious High-Achievers.
Predictive capability model	Technical practices + culture predict high DORA performance	- Research identifies capabilities driving outcomes - not a prescriptive blueprint but evidence-based practices.
Continuous evolution	2023: MTTR renamed FDRT 2024: Deployment Rework Rate added 2025: AI focus	- Framework adapts over time based on new research - the 2024 addition of Deployment Rework Rate was the most significant structural change since 2014.
Cross-industry validation	Patterns hold across finance, tech, healthcare, government	- Research findings generalize broadly - same metrics predict success regardless of industry or organization size.

Aspect	Example	Description
DORA Quick Check	Five questions at dora.dev/quickcheck – results in under one minute	• Free benchmark tool comparing team to industry performance tier • no data stored; instant result; useful for starting the DORA conversation with leadership.

TABLE 16

Trend Analysis Over Time

Single metric snapshots are almost meaningless — DORA metrics only become actionable when tracked as trends over weeks and quarters, with clear before/after comparisons tied to specific changes in process or tooling.

Pattern	Example	Description
Rolling averages	30-day moving average smooths weekly spikes	• Use time windows to reduce noise • daily volatility obscures meaningful trends.
Week-over-week comparison	This week DF: 12 deploys Last week: 9 = +33% improvement	• Short-period comparisons reveal recent changes • useful for evaluating specific interventions.
Quarter-over-quarter	Q1 median LT: 3 days Q4: 1 day = 67% improvement	• Longer period trends show sustained progress • smooths seasonal effects like holidays.
Seasonality adjustment	December shows lower DF due to code freeze	• Account for predictable variations • year-end freezes, major releases, and on-call rotations affect metrics.
Before/after intervention	Pre-automation: 5 day LT Post-automation: 1 day LT	• Measure impact of changes • compare periods before and after process improvements or tool adoption.
Regression detection	Alert when 7-day CFR exceeds 20% threshold	• Automated alerts for metric degradation • catch performance declines before they become systemic.

TABLE 17

Qualitative vs Quantitative Indicators

DORA numbers show what is happening in a delivery system; qualitative data explains why and whether the people inside that system are experiencing it sustainably. Both are needed for a complete picture.

Type	Example	Description
Quantitative DORA	Deployment Frequency: 3.2 per day	• Numerical measurements of delivery performance • objective, comparable, and trackable over time.
Qualitative surveys	"Deployments feel risky" survey response	• Subjective experience of development process • captures fear, toil, and satisfaction that numbers miss.
Developer experience	"Build pipeline is slow and frustrating"	• Team member perceptions and feelings • low satisfaction despite good metrics suggests hidden problems.
Psychological safety	Team discusses failures openly without blame	• Cultural health indicator • Westrum generative culture correlates with high DORA performance.
Complementary data	Combine CFR (quant) with incident retro quality (qual)	• Both types together provide complete picture • numbers show what happened, feedback explains why.
SPACE framework	Satisfaction, Performance, Activity, Communication, Efficiency	• Broader context for DORA • Microsoft SPACE model supplements delivery metrics with wellbeing and collaboration.
DX Core 4	Speed + Effectiveness + Quality + Business Impact	• Unified framework encapsulating DORA, SPACE, and DevEx into four counterbalanced dimensions • optimizing one dimension cannot improve the full score without also improving the others.

TABLE 18

Tools and Platforms

The DORA metrics tooling market has matured considerably, ranging from open-source self-hosted options like Apache DevLake to enterprise intelligence platforms; the right choice depends on your toolchain, team size, and whether you need DORA alone or a broader engineering intelligence stack.

Tool	Example	Description
Datadog DORA	Native dashboards with automatic deployment detection	• Observability platform with built-in DORA tracking • integrates CI/CD, APM, and incident data.
GitLab DORA	Native metrics for GitLab CI/CD pipelines	• Platform-native tracking for GitLab teams • automatic calculation from pipeline and incident data.
Apache DevLake	Open-source self-hosted: connects GitHub + Jira + Jenkins → Grafana dashboard	• Strongest open-source option; ingests 40+ DevOps tools • full control, no vendor lock-in; requires self-hosting and setup investment.
LinearB	Engineering intelligence platform with DORA benchmarks	• Specialized DORA tool connecting Git, Jira, and CI/CD • automatic metric calculation and team comparisons.
Faros AI	Data integration platform normalizing metrics across tools	• Engineering intelligence aggregating data from multiple sources • supports custom metric definitions.
Sleuth	Deployment tracking with automatic failure detection	• Lightweight DORA tracker focused on deployment frequency and CFR • simple GitHub integration.
GitHub Actions DORA	Open-source action calculating lead time from workflow logs	• Custom GitHub Action for teams building own DORA tracking • transparent calculation logic.
Four Keys (Google)	Open-source reference implementation on GCP	• Google's DORA reference tool demonstrating metric collection patterns • educational starting point for custom builds.

TABLE 19

Platform Engineering Impact

Platform engineering directly improves DORA metrics by eliminating the manual handoffs, waiting queues, and cognitive overhead that inflate lead time and reduce deployment frequency.

Practice	Example	Description
Internal Developer Platform	Self-service deployment portal with golden paths	• Platform teams enable developers to ship faster • standardized workflows reduce cognitive load and lead time.
Golden paths	Template repos with CI/CD, monitoring pre-configured	• Paved roads making the correct approach easiest • new services inherit high-quality patterns improving DORA metrics.
Self-service infrastructure	Developers provision databases via UI without tickets	• Eliminate waiting for IT operations • reduces lead time by removing manual handoffs.
Platform KPIs	Track platform adoption rate alongside DORA metrics	Platform team measures developer satisfaction and adoption plus downstream impact on delivery performance.
Developer experience	Survey: "How easy to deploy?" + measure actual DF/LT	• Combine perception and reality metrics • great platform should improve both satisfaction and delivery performance.
Platform as AI foundation	High-quality internal platforms amplify AI benefit at organizational scale	• 2025 DORA research: quality internal platforms are prerequisite for AI ROI • AI adoption without a robust platform produces disconnected local gains, not organizational improvement.

TABLE 20

Organizational Culture and Westrum Model

Culture is not a soft factor in DORA research — it is one of the strongest predictors of delivery performance. Westrum's three-category model provides a practical language for diagnosing organizational information flow and its impact on DORA outcomes.

Culture	Example	Description
Pathological	Information hoarded, messengers shot, failure hidden	• Power-oriented culture where information is political currency • consistently correlates with low DORA performance.
Bureaucratic	Rule-oriented, narrow focus, failure leads to blame	• Process-focused culture where rules matter more than mission • typical of medium DORA performance organizations.
Generative	Mission-oriented, cooperation, failure leads to inquiry	• Performance-oriented culture with high trust and information flow • strong predictor of elite DORA metrics.
Psychological safety	Team members speak up without fear of punishment	• Foundation for high performance • enables honest incident review and learning from failures, improving FDRT and CFR.
Blameless culture	Postmortems focus on system improvements not individual blame	• Learning-focused incident response • openness about failures improves both FDRT (faster recovery) and CFR (fewer repeat failures).
Information flow	Generative orgs share safety signals broadly and actively	Westrum's research shows how organizations process information predicts safety and performance outcomes.

TABLE 21

Advanced Topics and Extensions

As DORA matures and AI reshapes software development, the framework is extended with metrics that capture AI-era concerns — code durability, AI vs. human work attribution, and developer well-being — that the original four metrics cannot surface.

Topic	Example	Description
Deployment rework rate measurement	$3 \text{ unplanned incident-fix deploys} \div 20 \text{ total deploys} = 15\% \text{ rework rate}$	• Official 5th metric since 2024: count deployments triggered by production incidents as a share of all deployments • no official benchmarks yet; track directionally and set internal thresholds.
Value stream mapping	Visualize flow from idea → production including all wait states	• Map entire delivery process to identify bottlenecks • DORA metrics measure outcomes; VSM reveals causes.
SPACE framework integration	Combine DORA with Satisfaction, Activity, Communication	• Broader performance model adding wellbeing and collaboration • DORA alone doesn't capture developer experience or communication quality.
DX Core 4 framework	Speed + Effectiveness + Quality + Business Impact	• Unifies DORA, SPACE, and DevEx into four counterbalanced dimensions • adopted at 300+ companies including Meta, Microsoft, and Uber; prevents optimizing one dimension by hiding another.
AI-assisted development impact	90% of developers use AI; throughput up, stability down	• 2025 DORA finding: AI adoption correlates positively with throughput but negatively with delivery stability • AI accelerates code generation → larger batches → slower review → more instability.
Code turnover rate	6% of committed code rewritten within 14 days	• AI-era quality signal: measures code durability that CFR cannot see • AI code passing tests at deploy time may still be silently replaced within weeks; pre-AI baseline ~3.3%, rising with heavy AI use.
Service-level objectives	99.9% uptime SLO with error budget	• SRE practices complement DORA • error budgets and reliability budgets provide operational context alongside delivery metrics.

Topic	Example	Description
Regulatory constraints	Healthcare orgs face compliance overhead affecting lead time	• Industry-specific factors influence achievable benchmarks • compare against organizations with similar regulatory environments, not global averages.

TABLE 22

DORA AI Capabilities Model

The 2025 DORA AI Capabilities Model identifies seven practices that amplify the benefits of AI adoption in engineering organizations. These are established engineering best practices that become more critical — not less — when AI accelerates code generation and expands batch sizes.

Capability	Example	Description
User-centric focus	All AI coding effort tied to a specific user problem or job-to-be-done	• Most critical capability: without user focus, AI adoption can have a net-negative impact on team performance • AI makes building the wrong thing faster than ever before.
Quality internal platforms	Self-service deployment, automated testing, standardized pipelines	• Platform is the distribution layer for scaling AI benefits from individual to organizational • without a quality platform, AI gains remain isolated and disconnected from delivery outcomes.
Clear and communicated AI stance	Published policy: permitted tools, use cases, and data privacy rules	• Ambiguity about acceptable AI use stifles adoption and creates risk • organizations with clear AI policies show 451% higher team AI adoption than those without.
Working in small batches	Break AI-generated work into reviewable increments, not giant PRs	• Amplifies AI's positive influence on product performance • AI can generate large amounts of code rapidly; small batches keep review and testing manageable.
Healthy data ecosystems	Internal data is high-quality, accessible, and unified	• AI is only as good as its data • high-quality unified internal data substantially amplifies AI's positive influence on organizational performance.
AI-accessible internal data	AI tools connected to internal codebase, docs, and architecture diagrams	• Connecting AI to company-specific context transforms it from generic assistant to specialized tool • amplifies individual effectiveness and code quality.
Strong version control practices	Frequent commits, disciplined rollback use, short-lived feature branches	• AI increases code velocity, making the safety net of version control more critical • frequent commits and rollback proficiency amplify AI's positive impact on team effectiveness.

TABLE 23

AI-Era DORA Extensions

When AI tools generate 30–70% of committed code, standard DORA metrics become ambiguous — deployment frequency can inflate without meaningful productivity gain, and lead time can drop simply because coding accelerated. These extensions help teams interpret DORA accurately in AI-assisted environments.

Metric	Example	Description
AI code share	35% of merged code flagged as AI-assisted this sprint	• Segmentation layer for all other DORA metrics: what share of committed code was AI-generated? • without this, rising DF or falling LT is ambiguous — it could reflect AI inflation rather than pipeline improvement.
AI vs. human PR cycle time	AI-assisted PRs: avg 3.2 day review vs human PRs: 1.8 day review	• If AI-assisted PRs take longer to review, the bottleneck has shifted from code creation to code review • total lead time may still fall while review quality and cycle time worsens.
Code churn / code turnover rate	8% of AI-generated code rewritten within 14 days	• Measures code durability that CFR cannot capture: code that deploys successfully but is silently replaced within weeks • pre-AI baseline ~3.3%; values above 7% indicate significant engineering waste.

Metric	Example	Description
AI suggestion acceptance rate	Copilot acceptance rate: 32% today, was 44% three months ago	• Declining trend signals trust or relevance problems with AI output • 39% of developers trust AI outputs "a little" or "not at all" per 2025 DORA research.
PR review load per senior engineer	Senior engineers reviewing 40 PRs/week, up from 18 pre-AI	• Leading indicator of burnout and review degradation in AI-assisted teams • AI creates more code than review capacity can absorb, concentrating burden on senior staff.
Innovation rate	New feature work: 42% of time; bug/maintenance: 58%	• Ratio of time on new features vs reactive maintenance • if AI increases velocity but innovation rate declines, the team is faster on a treadmill, not delivering more value.

References

Official Documentation

1. DORA Metrics — Official Guide
 3. DORA Research Publications
 5. DORA Capabilities Overview
 7. DORA — Platform Engineering Capability
 9. DORA — Trunk-Based Development
 11. DORA — Monitoring and Observability
 13. DORA — Team Experimentation
 15. DORA 2025 AI Report: State of AI-Assisted Software Development
 17. Google Cloud: Introducing DORA's Inaugural AI Capabilities Model
 19. Google SRE Book: Service Level Objectives
 21. Datadog DORA Metrics Documentation
 23. GitHub: Lead Time for Changes Action
2. DORA Metrics History
 4. DORA Quick Check Tool
 6. DORA — Generative Organizational Culture
 8. DORA — Continuous Delivery Capability
 10. DORA — Continuous Integration
 12. DORA — Test Automation
 14. DORA 2025 State of DevOps Report
 16. Google Cloud: Announcing the 2025 DORA Report
 18. Google Cloud: Using the Four Keys to Measure DevOps Performance
 20. GitLab DORA Metrics Documentation
 22. Apache DevLake DORA Documentation

Technical Blogs & Tutorials

24. Accelerate: The Science of Lean Software — IT Revolution
 26. Atlassian: DORA Metrics Guide
 28. Faros AI: 2025 DORA Report Takeaways
 30. Swarmia: Engineering Metrics for Leaders
 32. GetDX: Measuring Developer Productivity with the DX Core 4
 34. GetDX: DORA Tools
 36. Honeycomb: DORA and Observability in 2025
 38. Axify: Understanding DORA Metrics Complete Guide
 40. Oobeya: DORA Metrics Not Enough 2026
 42. GitKraken: AI Impact on Software Development Guide
 44. GitDailies: DORA Metrics Complete Guide
 46. Keploy: Improve DORA Metrics
 48. CodeAnt: Top DORA Metric Tools
 50. MStone: Top Platform Engineering Metrics
 52. PlatformEngineering.org: Measure Developer Productivity and Platform ROI
 54. Planview: Measuring Key DevOps Metrics
 56. DORA Quick Check — Google
 58. Lucaberton: Developer Experience Metrics
 60. Improvado: What Are Qualitative Metrics
 62. RealVNC: DevOps Trends
 64. DevOps AI Decoded: Essential SRE Metrics 2026
 66. PracticalEngineering.Management: Westrum's Organizational Culture
 68. Excella: 3 Types of Culture
 70. Thoughtworks: Continuous Delivery Guide
 72. Martin Fowler: Continuous Delivery
 74. Martin Fowler: Deployment Pipeline
 76. Martin Fowler: Canary Release
 78. DORA: Improving Deployment Frequency
 80. Google Cloud: DevOps Measurement Guide
25. Westrum's Organizational Model in Tech Orgs — IT Revolution
 27. Faros AI: All You Need to Know About DORA Metrics
 29. Swarmia: DORA Metrics Guide
 31. GetDX: DORA Metrics Guide
 33. GetDX: Qualitative vs Quantitative Metrics
 35. LinearB: DORA Metrics Tools
 37. Axify: State of DevOps 2025
 39. Larridin: DORA Metrics Explained — Complete Guide 2026
 41. Splunk: State of DevOps 2025
 43. Plandek: DORA Metrics Guide
 45. Selleo: Software Delivery Performance
 47. Gotopia.tech: DORA and AI Integration
 49. Jellyfish: Platform Engineering Metrics
 51. MStone: Engineering Metrics Benchmarks for High Performing Teams
 53. PlatformEngineering.com: Metrics That Matter
 55. Sourcegraph: Engineering Metrics What Actually Matters in 2026
 57. RedMonk: DORA 2025 Report Analysis
 59. ConfluxHQ: Financial Value of Psychological Safety
 61. Tianpan: DORA Metrics in 2026 Forum
 63. GraphApp: Quantitative vs Qualitative Metrics
 65. Motadata: DevOps Metrics
 67. PsychSafety: Westrum's Cultural Typologies
 69. Medium: Organisational Culture and Software Delivery Performance
 71. Martin Fowler: Continuous Integration
 73. Martin Fowler: Feature Toggles
 75. Martin Fowler: Blue-Green Deployment
 77. Martin Fowler: Trunk-Based Development
 79. DORA: Improving Lead Time
 81. InfoQ: DORA Metrics Guide

- 82. ThoughtWorks Technology Radar: DORA Metrics
- 84. LinkedIn Engineering: Measuring Software Delivery
- 86. Spotify Engineering: Engineering Effectiveness
- 88. LaunchDarkly: Feature Management and DORA
- 90. Cortex: Engineering Scorecard
- 92. DX Platform: Engineering Metrics
- 94. Rootly: Incident Management and DORA
- 96. Dynatrace: DORA Metrics Automation
- 98. Opsgenie: On-Call and DORA
- 100. Codefresh: CI/CD Pipeline Metrics

Academic Papers

- 101. Forsgren, N., Humble, J., Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps (full text)
- 103. Greiler, M. et al. (2022). Code Ownership and Software Quality — ICSE
- 105. Kim, G., Humble, J., Debois, P., Willis, J. (2016). The DevOps Handbook
- 107. Google DORA Research: How to Transform
- 102. Forsgren, N. et al. (2021). The SPACE of Developer Productivity — ACM Queue
- 104. Westrum, R. (2004). A Typology of Organisational Cultures — BMJ Quality & Safety
- 106. Forsgren, N., Kersten, M. (2018). DevOps Metrics — ACM
- 108. Luzardo, G. et al. (2023). ROI of DevOps Transformations — Google Research

GitHub Repositories & Code Examples

- 109. DORA Four Keys — Google GitHub
- 111. Apache DevLake — GitHub
- 113. Keptn DORA Metrics — GitHub
- 115. LinearB Engineering Metrics — GitHub
- 110. Lead Time for Changes — GitHub Action
- 112. DORA Metrics Calculator — Open Source
- 114. Backstage Metrics Plugin — GitHub

Video Resources

- 116. DORA: How to Measure DevOps Performance — Google Cloud Tech (YouTube)
- 118. State of DevOps 2024 — DORA Team (YouTube)
- 120. Jez Humble: Continuous Delivery (YouTube)
- 122. Google Cloud DORA 2025 Keynote (YouTube)
- 117. DORA Metrics Explained in 10 Minutes (YouTube)
- 119. Nicole Forsgren: Accelerate — DevOpsDays (YouTube)
- 121. DORA Quick Check Walkthrough (YouTube)
- 123. Measuring Software Delivery Performance with DORA — GOTO Conference (YouTube)

Industry Best Practice Guides & Books

- 124. The DevOps Handbook, 2nd Edition — Gene Kim et al.
- 126. Team Topologies — Matthew Skelton & Manuel Pais
- 128. Site Reliability Engineering — Google
- 130. DORA 2023 State of DevOps Report — Google Cloud
- 132. DORA 2021 State of DevOps Report — Google Cloud
- 134. Lean Enterprise — Jez Humble et al.
- 136. Flow Engineering — Steve Pereira & Andrew Davis
- 125. Project to Product — Mik Kersten
- 127. Continuous Delivery — Jez Humble & David Farley
- 129. DORA 2024 State of DevOps Report — PDF
- 131. DORA 2022 State of DevOps Report — Google Cloud
- 133. The Phoenix Project — Gene Kim
- 135. Making Work Visible — Dominica DeGrandis

Other

- 137. DORA Community — Google Groups
- 139. DevOps Research and Assessment — Homepage
- 141. Google Cloud DevOps Best Practices
- 143. Puppet: State of DevOps Report Archive
- 145. DevOps Institute: Upskilling Report
- 147. InfoQ: DevOps and Continuous Delivery
- 149. The New Stack: Platform Engineering
- 151. Engineering Enablement Podcast: DORA
- 153. Software Engineering Radio: DevOps Metrics
- 155. Practical AI for Engineering Leaders
- 157. Shopify Engineering: Deployment Velocity
- 159. Amazon Builders Library: Deployment Safety
- 161. LaunchDarkly: Feature Flag Best Practices
- 163. Harness: DORA Metrics with CI/CD
- 165. CNCF Platforms White Paper
- 167. DX Core 4 Research Report — GetDX
- 138. State of DevOps Survey — Annual
- 140. DORA Report Archive — All Years
- 142. PagerDuty: State of Digital Operations
- 144. CNCF: Cloud Native DevOps
- 146. ThoughtWorks: Technology Radar
- 148. DZone: DevOps Guides
- 150. LeadDev: Engineering Effectiveness
- 152. Changelog: DORA and Software Delivery
- 154. Greater Than Code: Developer Experience
- 156. Stripe Engineering: Deployment Infrastructure
- 158. Etsy Code as Craft: Deployment Metrics
- 160. Honeycomb: Observability-Driven Development
- 162. Split.io: Progressive Delivery Guide
- 164. GitOps Working Group: Deployment Practices
- 166. Platform Engineering Community
- 168. Oobeya: Engineering Intelligence Platform

169. Waydev: Engineering Analytics

170. Propelo: Engineering Insights and DORA